Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Programming Project 1

Solving 8-Puzzle Problem Using A* Search Algorithm

Problem Formulation

The 8-puzzle is a sliding tile puzzle played on a 3x3 grid containing 8 numbered tiles from 1-8, and one blank space (often displayed as 0). A tile adjacent to the blank can slide into it. The objective is to reach a specified goal arrangement of the tiles using the fewest moves possible. This project assignment formulates the puzzle as a search problem and solves it using the A* algorithm approach.

State Representation

Each state of the puzzle is represented as a flat list of 9 integers, and read row by row from top-left to bottom-right. The value 0 represents the blank tile. For example, the board below:

1	2	3
4	5	6
7		8

Is stored internally as a list -> [1, 2, 3, 4, 5, 6, 7, 0, 8]. The index ‘i’ if an entry in the list maps to a (row, col) grid position via **row = i // 3** and **column = i % 3**.

Initial and Goal States

Both the initial and goal states are supplied by the user at run time (when the user runs the Python file in the terminal, they will be prompted to enter user inputs for the initial and goal states). It is done through calling the **function read_state**. The program validates that the input contains exactly the digits 0-8 with no repeats before accepting it. Any permutations may be entered as the goal state, not only the conventional 1-8 blank ordering, which is why the goal is represented generically rather than hard-coded.

Operators/Actions

There are four possible operators, corresponding to sliding the blank tile Up, Down, Left, or Right. Moving the blank is equivalent to moving the neighboring tile in the opposite direction. Because the board is stored as a flat list, each operator is implemented as a fixed offset added to the blank's index. Each operator has a step cost of 1, and every state has at most 4 legal operators.

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Move	Blank Index	Illegal when
U (up)	Index - 3	Index < 3 (Blank is in row 0)
D (down)	Index + 3	Index > 5 (Blank is in row 2)
L (left)	Index - 1	Index % 3 == 0 (Blank is in column 0)
R (right)	Index + 1	Index % 3 == 2 (Blank is in column 2)

Path Cost (g(n))

g(n) is the number of moves taken to reach a state 'n' from the initial state. It is the depth of 'n' in the search tree. Since every move costs 1, g(n) simply equals the depth counter carried by each node, incremented by 1 each time a child state is generated.
For example, if a state requires five moves to reach from the initial state, g(n) = 5.

Heuristic Function (h(n))

Two admissible heuristics are implemented and compared:

1. **h1 -> Misplaced Tiles:** counts the number of non-blank tiles that are not in their goal position. It never overestimates the true cost because every misplaced tile needs at least one move.
2. **h2 -> Manhattan Distance:** sums, over every tile which isn't blank, the number of grid rows plus columns separating its current position from its goal position. It is also admissible, and it dominates h1 ($h2(n) \geq h1(n)$ for every state), which makes it a more informed and generally a more efficient heuristic. For each tile: **Manhattan Distance=$|$ x current − x goal $|$ + $|$y current − y goal $|$**

A* algorithm evaluates each node with **f(n) = g(n) + h(n)** and always expands the node in the priority queue with the lowest f(n) value, guaranteeing an optimal (shortest) solution as long as h(n) is admissible (one that never overestimates the true cost of reaching the goal from any given node).

Program Structure

The program is written in Python 3 and uses the built-in **heapq** module to implement the A* priority queue (open list) as a binary min-heap.

Global Data Structures

1. **moves:** a dictionary mapping each operator label ('U', 'D', 'L', 'R') to its index offset (-3, +3, -1, +1)

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

2. **HEURISTICS:** a list of (name, function) pairs for the main program to be able to loop over both heuristics without duplicating code.

Class: PuzzleState

This class represents one node of each search graph/tree. It stores the board configuration, a reference to its parent node (for backtracking the solution path), the move that produced it, its depth (g-cost), and its total estimated cost f(n) = g(n) + h(n). It implements **__lt__** function so that Python's heapq can order nodes by cost automatically.

Functions and Procedures

Function	Purpose
print_board(board)	Prints a 3×3 ASCII grid for a given board (display only).
heuristic_misplaced(board, goal_pos)	Computes h1: count of tiles not in their goal position.
heuristic_manhattan(board, goal_pos)	Computes h2: sum of Manhattan distances of all tiles from their goal position.
move_tile(board, move, blank_pos)	Returns a new board with the blank swapped in the given direction.
a_star(start_state, goal_state, heuristic)	Core search: pops the lowest-cost node from the open list, checks for the goal, expands legal children, and pushes them back with updated g and f costs. Tracks nodes generated (pushed) and expanded (popped/processed) and returns the goal node plus both counters.
print_solution(solution)	Walks the parent chain from the goal node back to the root, reverses it, and prints the move-by-move board sequence.
read_state(prompt)	Reads and validates a 9-value board from the user (rejects non-numeric input or any input that isn't a permutation of 0–8).
inversion_parity(board)	Counts tile inversions (ignoring the blank) and returns the count mod 2, used for the solvability pre-check.

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Not every pair of (initial, goal) states is reachable from one another. Sliding a tile never changes the parity of the number of inversions in the board (with the blank ignored). Before running A*, the program computes inversion_parity for both the initial and goal states; if they differ, it reports “No solution exists” immediately instead of searching, which correctly predicts unsolvable inputs such as Case 7 below.

Overall Algorithm Flow

1. Read and validate the user’s input for initial and goal states from the user.
2. Check solvability via inversion_partity; stop early if unsolvable.
3. For each heuristic (h1, h2), run A* from scratch and record nodes generated, nodes expanded, and the resulting path
4. Print the move-by-move solution and the two node-count statistics for each heuristic.

Eight Test Cased and Results

Each case below was run once per heuristic. The move sequence lists the direction the blank tile was slid at each step (U = up, D = down, L = left, R = right).

Case 1

Initial State

1	**2**	**3**
7	**4**	**5**
6	**8**	

Goal State

1	**2**	**3**
8	**6**	**4**
7	**5**	

h1: Misplaced Tiles

Move sequence: U → L → D → L → U → R → D → R
Path length (number of moves), g: 8

Nodes generated: 42

Nodes expanded: 21

h2: Manhattan Distance

Move sequence: U → L → D → L → U → R → D → R
Path length (number of moves), g: 8

Nodes generated: 19

Nodes expanded: 9

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Case 2

Initial State

2	**8**	**1**
3	**4**	**6**
7	**5**	

Goal State

3	**2**	**1**
8		**4**
7	**5**	**6**

h1: Misplaced Tiles

Move sequence: U → L → U → L → D → R
Path length (number of moves), g: 6

Nodes generated: 15

Nodes expanded: 7

h2: Manhattan Distance

Move sequence: U → L → U → L → D → R
Path length (number of moves), g: 6

Nodes generated: 13

Nodes expanded: 6

Case 3

Initial State

7	**2**	**4**
5		**6**
8	**3**	**1**

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

h1: Misplaced Tiles

Move sequence: D → R → U → L → L → U → R → R → D → L → D → L → U → R → U → L → D → R → R → D
Path length (number of moves), g: 20

Nodes generated: 6025

Nodes expanded: 3653

h2: Manhattan Distance

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Move sequence: D → R → U → L → L → U → R → R → D → L → D → L → U → R → U → L → D → R → R → D
Path length (number of moves), g: 20

Nodes generated: 443

Nodes expanded: 270

Case 4

Initial State

1	**2**	**3**
4		**6**
7	**5**	**8**

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

h1: Misplaced Tiles

Move sequence: D → R
Path length (number of moves), g: 2

Nodes generated: 7

Nodes expanded: 2

h2: Manhattan Distance

Move sequence: D → R
Path length (number of moves), g: 2

Nodes generated: 7

Nodes expanded: 2

Case 5

Initial State

4	**1**	**3**
7	**2**	**5**
	8	**6**

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

h1: Misplaced Tiles

Move sequence: U → U → R → D → R → D

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Path length (number of moves), g: 6

Nodes generated: 13

Nodes expanded: 6

h2: Manhattan Distance

Move sequence: U → U → R → D → R → D
Path length (number of moves), g: 6

Nodes generated: 13

Nodes expanded: 6

Case 6

Initial State

8	**6**	**7**
2	**5**	**4**
3		**1**

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

h1: Misplaced Tiles

Move sequence: R → U → U → L → D → D → R → U → U → L → D → L → D → R → R → U → L → L → U → R → R → D → L → L → D → R → U → U → R → D → D
Path length (number of moves), g: 31

Nodes generated: 179139

Nodes expanded: 122286

h2: Manhattan Distance

Move sequence: R → U → U → L → L → D → D → R → U → U → L → D → D → R → R → U → L → U → L → D → D → R → R → U → U → L → D → L → D → R → R
Path length (number of moves), g: 31

Nodes generated: 11619

Nodes expanded: 7396

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Case 7 (Unsolvable)

Initial State

1	**2**	**3**
4	**5**	**6**
8	**7**	

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

Inversion parity of the initial state differs from that of the goal state → the program correctly reports no solution exists without running A*.

Case 8

Initial State

2	**3**	**6**
1	**5**	**8**
4		**7**

Goal State

1	**2**	**3**
4	**5**	**6**
7	**8**	

h1: Misplaced Tiles

Move sequence: R → U → U → L → L → D → D → R → R
Path length (number of moves), g: 9

Nodes generated: 22

Nodes expanded: 12

h2: Manhattan Distance

Move sequence: R → U → U → L → L → D → D → R → R
Path length (number of moves), g: 9

Nodes generated: 16

Nodes expanded: 9

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Summary Table

The table summarizes the path length and search effort for both heuristics across all eight test cases.

Case	Heuristic	Moves (g)	Nodes Generated	Nodes Expanded
Case 1	h1: Misplaced Tiles	8	42	21
Case 1	h2: Manhattan Distance	8	19	9
Case 2	h1: Misplaced Tiles	6	15	7
Case 2	h2: Manhattan Distance	6	13	6
Case 3	h1: Misplaced Tiles	20	6025	3653
Case 3	h2: Manhattan Distance	20	443	270
Case 4	h1: Misplaced Tiles	2	7	2
Case 4	h2: Manhattan Distance	2	7	2
Case 5	h1: Misplaced Tiles	6	13	6
Case 5	h2: Manhattan Distance	6	13	6
Case 6	h1: Misplaced Tiles	31	179139	122286
Case 6	h2: Manhattan Distance	31	11619	7396
Case 7	—	No solution	—	—
Case 8	h1: Misplaced Tiles	9	22	12
Case 8	h2: Manhattan Distance	9	16	9

Group Names: Krishna Patel and Prithvi Koka
Course: ITCS 6150-091
Programming Language: Python 3

Conclusion

Across every solvable case, both heuristics returned the same optimal path length (as A* with admissible heuristic guarantees), but Manhattan distance (h2) consistently expanded and generated the same number of nodes or fewer than the misplaced-tiles heuristic (h1), never more. The gap grows sharply with puzzle difficulty: on the easy cases (ex: Case 4, 2 moves) both heuristics performed identically, but on the hardest case in this set, Case 6 (a known worst-case 31-move instance), h1 expanded 122,286 nodes while h2 expanded only 7,396. This matches the theoretical expectation that Manhattan distance dominates misplaced tiles ($h2(n) \geq h1(n)$ for all n), giving A* a tighter, more informative estimate of the remaining cost and letting it prune far more of the search space while still finding the optimal solution. Case 7 also confirms that the inversion-parity check correctly identifies unsolvable instances before any search is attempted, avoiding wasted computation.